

On-Policy vs. Off-Policy Reinforcement Learning for Autonomous Racing in F1TENTH

Mayank Goud Polagoni Kushwanth Vellanki
College of Engineering, Northeastern University, Boston, MA
{polagoni.m, vellanki.ku}@northeastern.edu

ABSTRACT

Autonomous racing presents a compelling testbed for reinforcement learning (RL) in safety-critical continuous control domains. In this work, we conduct a systematic empirical comparison of three deep RL algorithms, Proximal Policy Optimization (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3), against classical baselines (a reactive LiDAR controller and a Pure Pursuit controller) in the F1TENTH Gym simulator. Our evaluation spans five complementary dimensions: sample efficiency via learning curves, final on-track performance, multi-seed stability, generalization to an unseen track (Berlin), and an ablation study on reward shaping with a steering-magnitude penalty. Results demonstrate that off-policy methods, particularly SAC, substantially outperform both the on-policy PPO and classical controllers. SAC achieves a mean reward of 0.289 and average speed of 2.891 m/s compared to PPO’s 0.138 and 1.376 m/s. TD3 attains the highest instantaneous speed (3.0 m/s). Multi-seed analysis confirms SAC’s consistency, and generalization experiments reveal that SAC maintains robust performance on the unseen Berlin map while PPO degrades significantly. The ablation study shows that a steering penalty produces smoother trajectories at a modest cost to cumulative reward. These findings underscore the advantages of off-policy learning with entropy regularization for continuous autonomous racing.

1. INTRODUCTION

Autonomous racing has emerged as a prominent benchmark domain for reinforcement learning (RL) research, combining the challenges of continuous control, real-time decision-making, and safety-critical constraints within a single problem formulation. Unlike traditional control benchmarks such as the MuJoCo locomotion suite, autonomous racing demands that an agent simultaneously optimize for speed, which directly increases the probability of catastrophic failure through collision and safety, which imposes implicit constraints on the policy’s aggressiveness. This fundamental tension between performance and safety makes racing a particularly informative testbed for evaluating RL algorithm capabilities.

The F1TENTH platform, originally developed as a 1/10-scale physical racing car for academic competitions, has become a standardized simulation environment for RL research (O’Kelly et al.). The F1TENTH Gym simulator provides realistic Ackermann steering dynamics, simulated 2D LiDAR perception, collision detection, and multiple track layouts, enabling controlled and reproducible experimentation. Its Gymnasium-compatible interface facilitates direct integration with standard RL training pipelines.

In this project, we systematically compare three deep RL algorithms: PPO (Schulman et al., 2017), an on-policy method with clipped surrogate objectives; SAC (Haarnoja et al., 2018), an off-policy method incorporating maximum entropy regularization; and TD3 (Fujimoto et al., 2018), an off-policy deterministic actor-critic with twin critics and delayed updates. We evaluate these against two classical baselines: a reactive LiDAR-based heuristic controller and a Pure Pursuit geometric path-tracking controller. Our comparison is structured around five experimental axes: (1) sample efficiency, (2) final performance on the training track, (3) multi-seed stability, (4) generalization to an unseen track, and (5) an ablation study on reward function design.

The primary contributions of this work are: (a) a comprehensive, controlled comparison of on-policy versus off-policy deep RL for continuous autonomous racing, (b) empirical evidence that off-policy methods with entropy regularization are superior across all evaluation dimensions, (c) a demonstration that reward shaping via a steering penalty can modulate the speed–smoothness trade-off, and (d) a fully reproducible codebase with modular environment wrappers, training scripts, and evaluation pipelines.

2. BACKGROUND

2.1 Reinforcement Learning Formulation

We formalize autonomous racing as a Markov Decision Process (MDP) defined by the tuple (S, A, P, R, γ) , where S is the state space, A is the action space, $P(s' | s, a)$ represents the transition dynamics, $R(s, a)$ is the reward function, and $\gamma \in [0, 1)$ is the discount factor. The agent’s objective is to learn a policy $\pi(a | s)$ that maximizes the expected discounted return:

$$J(\pi) = E\pi \left[\sum_t \gamma^t R(s_t, a_t) \right]$$

2.2 Proximal Policy Optimization (PPO)

PPO (Schulman et al., 2017) is an on-policy policy gradient algorithm that constrains updates using a clipped surrogate objective. The probability ratio $r_t(\theta) = \pi_\theta(a_t | s_t) / \pi_{\theta_{\text{old}}}(a_t | s_t)$ is clipped to $[1-\epsilon, 1+\epsilon]$, preventing destructively large policy changes. PPO uses Generalized Advantage Estimation (GAE) with parameter λ to balance bias and variance. Being on-policy, PPO discards data after each update, limiting sample efficiency relative to off-policy methods.

2.3 Soft Actor-Critic (SAC)

SAC (Haarnoja et al., 2018) is an off-policy actor-critic algorithm that augments the standard RL objective with an entropy term $H(\pi(\cdot | s))$, encouraging exploration and preventing premature convergence. The modified objective is:

$$J(\pi) = \sum E[R(s_t, a_t) + \alpha H(\pi(\cdot | s_t))]$$

where α is the temperature parameter that controls exploration–exploitation balance. SAC maintains a replay buffer of past transitions and uses twin Q-networks with a separate target network for stable value estimation. The stochastic policy enables robust exploration in continuous control domains.

2.4 Twin Delayed DDPG (TD3)

TD3 (Fujimoto et al., 2018) extends the DDPG framework with three key modifications: (1) twin Q-networks, where the minimum of two critic estimates mitigates overestimation bias; (2) delayed policy updates, where the actor updates less frequently than the critics; and (3) target policy smoothing, where noise is added to target actions. TD3 learns a deterministic policy and relies on additive exploration noise during training.

2.5 Classical Baselines

We employ two non-learning baselines. The **Reactive LiDAR Controller** partitions the downsampled LiDAR scan into three sectors, computes steering from the left–right range differential, and sets speed heuristically based on frontal clearance. The **Pure Pursuit Controller** selects a lookahead point by maximum LiDAR range in the frontal sector, computes a heading error via the Pure Pursuit geometric formula, and applies proportional steering with clearance-modulated speed. Neither baseline adapts through experience.

3. RELATED WORK

Reinforcement learning for autonomous driving has been an active research area. PPO has been applied to various continuous control benchmarks and provides stable training,

though its on-policy nature limits sample efficiency (Schulman et al., 2017). SAC and TD3 were developed specifically to address sample efficiency and stability in continuous action spaces, with SAC’s entropy regularization proving effective for exploration in high-dimensional problems (Haarnoja et al., 2018; Fujimoto et al., 2018).

In the F1TENTH domain, Bosello demonstrated DQN with discretized actions for LiDAR-based racing, achieving functional but limited performance. The FT-Autonomous project applied PPO to the F1TENTH simulator. Koirala and Fleming (2024) explored offline RL methods for F1TENTH, showing that agents can learn from pre-collected datasets. Classical approaches such as Pure Pursuit (Coulter, 1992) and reactive gap-following controllers remain competitive baselines due to their simplicity and reliability. Our work bridges this gap by providing a controlled comparison between modern deep RL methods and classical approaches within a unified evaluation framework.

4. PROJECT DESCRIPTION

4.1 Environment and Simulator

All experiments are conducted in the F1TENTH Gym simulator, which models a 1/10-scale Ackermann-steered vehicle with realistic dynamics including tire slip, inertia, and collision physics. The simulator exposes a Gymnasium-compatible interface (`reset()/step()`) and returns structured observation dictionaries containing LiDAR scans, ego-motion states, collision flags, and lap timing information.

4.2 State Representation

The observation vector is 111-dimensional, consisting of 108 uniformly downsampled LiDAR beams (from 1080 raw beams spanning 270°) and 3 ego-motion features: forward velocity (v_x), lateral velocity (v_y), and yaw rate (ω_z).

Table 1: State representation.

Component	Dim	Description
LiDAR	108	Downsampled ranges
Ego	3	v_x, v_y, ω_z
Total	111	Concatenated vector

4.3 Action Space

The action space is continuous and two-dimensional: steering angle $\delta \in [-0.4, +0.4]$ radians and speed $u \in [0.0, 3.0]$ m/s.

4.4 Reward Function

The primary reward function incentivizes forward progress while penalizing collisions:

$$r(s, a) = 0.1 \times v_x - 10.0 \times \mathbb{I}[\text{collision}]$$

where v_x is forward velocity and the indicator function equals 1 upon collision, which also terminates the episode. For the ablation study (Section 6.6), we add a steering penalty:

$$rabl = 0.1 \cdot v_x - 0.1 \cdot |\delta| - 10.0 \cdot \mathbb{I}[\text{collision}]$$

4.5 Algorithm Configurations

We trained all three RL algorithms with MLP policy networks. PPO was trained for 20,000 timesteps and SAC and TD3 for 50,000 timesteps, reflecting the different data utilization patterns of on-policy versus off-policy methods.

Table 2: Hyperparameter configurations.

Parameter	PPO	SAC	TD3
Learning Rate	3e-4	3e-4	3e-4
Discount (γ)	0.99	0.99	0.99
Batch Size	64	256	256
Buffer Size	—	100K	100K
Timesteps	20K	50K	50K
n_steps	1024	1	1
n_epochs	10	—	—
Clip (ϵ)	0.2	—	—
GAE λ	0.95	—	—
Entropy	0.01	auto	—
τ (target)	—	0.005	0.005
Warmup	—	1000	1000

4.6 Baseline Implementations

Reactive LiDAR Controller.

We partition the 108-beam LiDAR scan into three equal sectors (left, center, right). Steering is the clipped difference of right and left mean ranges. Speed is set to 0.5 m/s when frontal clearance < 2.0 m, 1.0 m/s when < 5.0 m, and 2.0 m/s otherwise.

Pure Pursuit Controller.

We identify a lookahead point by selecting the maximum-range beam in the frontal 50% of the LiDAR scan. Using the vehicle’s pose (x, y, θ) , we compute heading error $\alpha = \arctan2(y_{\text{target}} - y, x_{\text{target}} - x) - \theta$ and apply proportional steering $\delta = \text{clip}(1.5\alpha, -0.4, 0.4)$ with a lookahead distance of 0.8 m. Speed is modulated by frontal clearance (0.8, 1.2, or 2.0 m/s).

4.7 Software Architecture

Our codebase is organized into modular components. The **F110GymWrapper** module handles raw observation processing, LiDAR downsampling, reward computation, and collision termination. The **F110SB3Env** module provides a Gymnasium-compliant interface with properly defined observation and action spaces. Separate training scripts configure and execute training with seed control, and

evaluation scripts produce trajectory plots, reward curves, speed profiles, and steering traces. For the ablation and generalization experiments, variant wrappers modify only the reward function or track map while preserving all other dynamics.

5. EXPERIMENTAL SETUP

5.1 Training Protocol

All RL models are trained on the default Levine track. PPO is trained for 20,000 timesteps and SAC/TD3 for 50,000 timesteps. Each run uses a fixed random seed for reproducibility. For the multi-seed stability analysis, PPO and SAC are each trained with three seeds (0, 1, 2).

5.2 Evaluation Protocol

Trained models are evaluated in deterministic mode for up to 1,000 steps. Metrics include: mean/max reward, mean/max speed, collision rate, steering smoothness (mean $|\Delta\delta|$), and trajectory shape. For generalization, models trained on Levine are deployed on the unseen Berlin track without fine-tuning.

Table 3: Evaluation metrics.

Metric	Definition
Avg Reward	Mean step reward over episode
Max Reward	Peak single-step reward
Avg Speed	Mean forward velocity (m/s)
Max Speed	Peak velocity (m/s)
Collision Rate	Fraction of collision terminations
Smoothness	$\text{mean}(\Delta\delta)$ over steps

6. RESULTS AND ANALYSIS

6.1 Classical Baseline Performance

Before analyzing RL methods, we establish the performance of the classical baselines. The reactive controller produces an oscillatory trajectory due to bang-bang steering: the left–right sector difference switches sign rapidly, resulting in zigzag motion. Despite this, the controller avoids collisions by adapting speed conservatively. Pure Pursuit produces smoother trajectories via geometric path tracking. However, both baselines are inherently conservative: the reactive controller averages 1.534 m/s and Pure Pursuit averages 1.958 m/s, well below the 3.0 m/s limit.

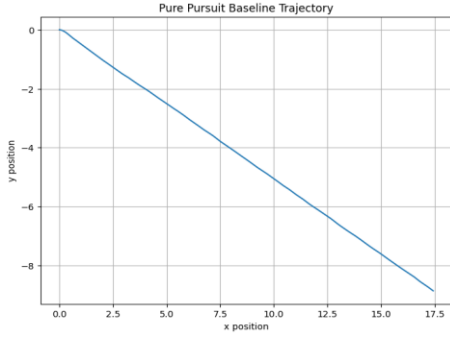


Figure 1: Pure Pursuit trajectory on the Levine training track.

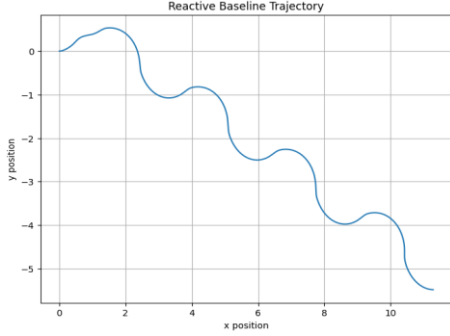


Figure 2: Reactive controller trajectory on the Levine training track.

6.2 Sample Efficiency Analysis

Figure 3 compares the learning curves of PPO and SAC, plotting mean episode reward versus environment steps.

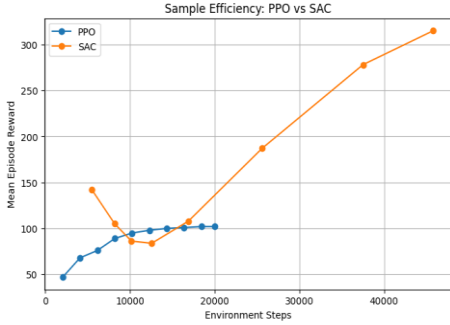


Figure 3: Sample efficiency. SAC (orange) achieves higher reward and faster improvement than PPO (blue).

SAC demonstrates markedly superior sample efficiency: it reaches approximately 150 reward by 10,000 steps and exceeds 300 by 45,000 steps. PPO plateaus around 100 by 6,000 steps with negligible improvement thereafter. This 3 $\times$ performance gap at convergence is attributable to SAC's replay buffer (enabling multiple gradient steps per interaction) and its entropy-regularized objective (maintaining diverse exploration).

6.3 Final Performance Comparison

Table 4 presents the comprehensive final performance comparison across all five methods on the Levine training track.

Table 4: Final performance on Levine track.

Method	Avg R	Max R	Avg S	Max S
Reactive	0.153	0.198	1.534	1.981
Pursuit	0.196	0.200	1.958	2.000
PPO	0.138	0.149	1.376	1.493
SAC	0.289	0.297	2.891	2.968
TD3	0.293	0.300	2.932	3.000

Off-policy dominance. SAC (Avg R = 0.289) and TD3 (Avg R = 0.293) dramatically outperform PPO (0.138) and the classical baselines. The off-policy methods achieve approximately 2 $\times$ the reward of the best classical baseline and 2.1 $\times$ PPO's reward.

PPO underperformance. PPO's average reward (0.138) falls below even the classical baselines. With only 20,000 timesteps, PPO has insufficient data to learn effective speed–steering coordination in this continuous domain.

Speed achievement. TD3 achieves the highest maximum speed (3.0 m/s), saturating the action space limit. SAC follows closely at 2.968 m/s.

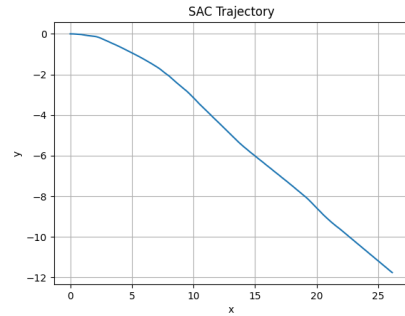


Figure 4: SAC trajectory on the training track.

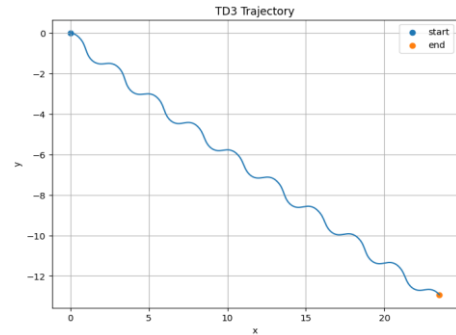


Figure 5: TD3 trajectory on the training track.

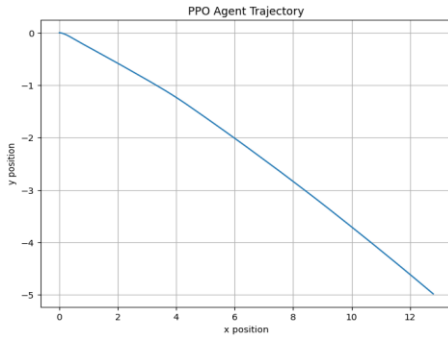


Figure 6: PPO trajectory on the training track.

SAC and TD3 produce smooth, extended trajectories navigating deep into the track, while PPO’s trajectory is shorter and more conservative, indicative of slower speeds and earlier collision termination.

6.4 Multi-Seed Stability Analysis

We evaluate PPO and SAC across three random seeds (0, 1, 2) to assess training robustness.

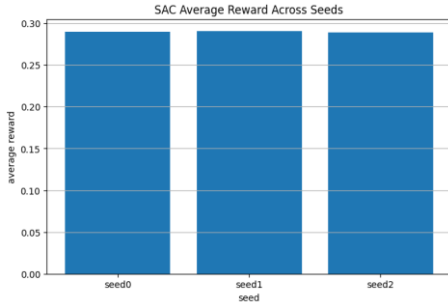


Figure 7: SAC average reward across seeds.

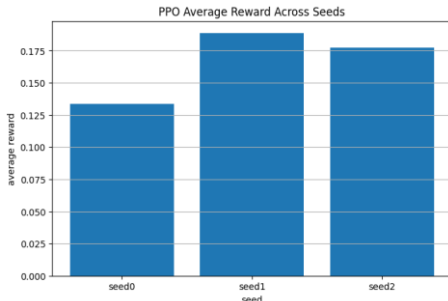


Figure 8: PPO average reward across seeds.

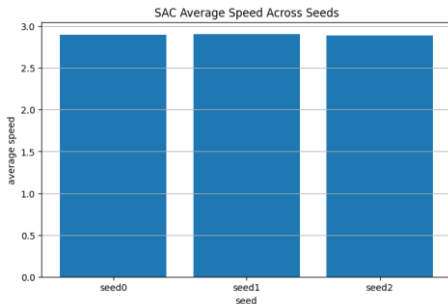


Figure 9: SAC average speed across seeds.

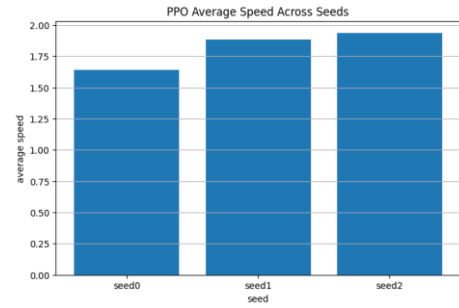


Figure 10: PPO average speed across seeds.

SAC exhibits substantially lower variance across seeds. All three SAC seeds converge to similar reward and speed profiles, indicating robust and reproducible training. PPO shows higher inter-seed variance, with some seeds stagnating near baseline levels, a known characteristic of on-policy methods in continuous control settings due to higher variance in stochastic policy gradient estimates.

6.5 Generalization to Unseen Track

We evaluate PPO and SAC models (trained on Levine) on the Berlin track without fine-tuning.

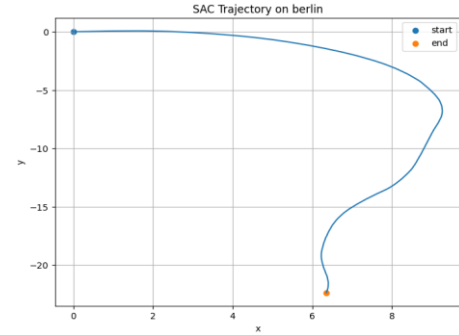


Figure 11: SAC trajectory on unseen Berlin track.

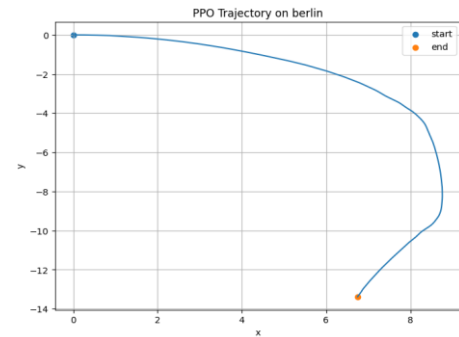


Figure 12: PPO trajectory on unseen Berlin track.

SAC demonstrates notably superior generalization, navigating substantially more of the Berlin track before termination while maintaining high speed. PPO collides early. This advantage stems from SAC’s entropy-regularized policy, which maintains broader behavioral coverage during training, yielding a policy less overfit to specific track geometry.

6.6 Ablation Study: Reward Shaping

We investigate the effect of reward design by training SAC with a modified reward that includes a steering-magnitude penalty ($\text{rabl} = 0.1 \cdot v_x - 0.1 \cdot |\delta| - 10 \cdot \mathbb{1}[\text{collision}]$).

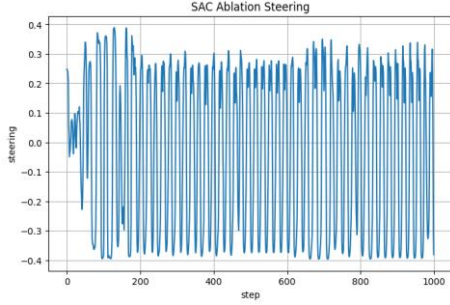


Figure 13: SAC ablation steering commands.

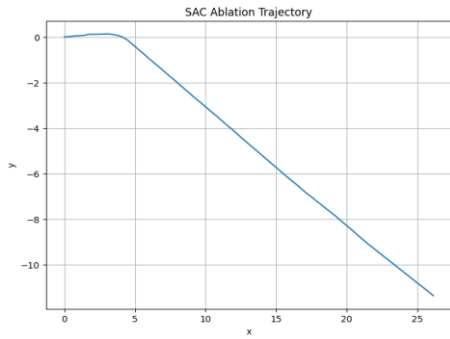


Figure 14: SAC ablation trajectory.

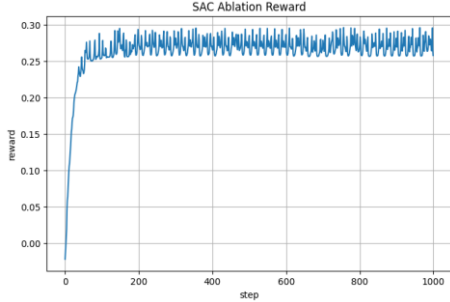


Figure 15: SAC ablation per-step reward.

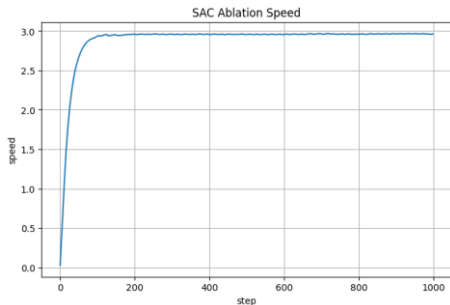


Figure 16: SAC ablation speed profile.

The ablation confirms the hypothesis: the steering penalty produces visibly smoother steering commands with reduced oscillation amplitude. However, this smoothness comes at a modest cost in cumulative reward. The trajectory

demonstrates competent navigation with smoother cornering. This experiment illustrates a controllable trade-off: reward shaping can modulate policy aggressiveness without architectural changes.

7. DISCUSSION

7.1 On-Policy vs. Off-Policy Trade-offs

The results clearly demonstrate that off-policy methods (SAC, TD3) are superior to on-policy PPO for continuous autonomous racing across all evaluation dimensions. The fundamental advantage stems from data reuse: SAC and TD3’s replay buffers allow each transition to inform multiple gradient updates, whereas PPO discards data after each epoch. In a domain where each step requires LiDAR simulation and collision checking, this sample efficiency advantage is practically significant.

Between the off-policy methods, SAC’s entropy regularization provides an additional advantage in exploration stability. The automatic temperature tuning ensures sufficient stochasticity to discover high-reward trajectories without excessive conservatism. TD3, while achieving marginally higher peak speed, relies on deterministic policies with additive noise, which can be less effective at discovering diverse behavioral modes.

7.2 Speed–Safety Trade-off

A central theme is the tension between speed and safety. Classical baselines represent the conservative extreme: safe but slow. Off-policy RL agents represent the aggressive end: fast with non-zero collision risk. The reward function mediates this trade-off through the ratio of speed incentive ($0.1 \cdot v_x$) to collision penalty (-10.0). Our ablation demonstrates that this balance can be further refined through additional reward terms without retraining from scratch.

7.3 Generalization and Robustness

The generalization experiment reveals that policies trained on a single track may not transfer reliably. SAC’s superior generalization to Berlin suggests entropy-regularized training produces policies with broader behavioral coverage, less overfit to specific environmental features. Future work could investigate domain randomization or multi-track training to improve generalization further.

7.4 Limitations

Several limitations should be acknowledged. First, the training budget (20K–50K steps) is modest; longer training could improve PPO’s relative performance. Second, we use a single training track, limiting generalization claims. Third, TD3 was evaluated with fewer seeds. Fourth, the 2D LiDAR

representation omits visual information available in physical racing. Finally, the simulator does not capture all real-world phenomena such as tire degradation, sensor noise, or actuator latency.

8. CONCLUSION

We conducted a systematic comparison of three deep RL algorithms, PPO, SAC, and TD3, against classical baselines for autonomous racing in the F1TENTH simulator. Our evaluation across five dimensions yields the following principal conclusions:

First, off-policy methods decisively outperform on-policy PPO in this continuous control domain. SAC achieves $2.1\times$ the average reward of PPO and $1.9\times$ the best classical baseline.

Second, SAC emerges as the most robust algorithm, demonstrating consistent multi-seed performance, superior generalization, and competitive final performance. Its entropy regularization provides a natural exploration mechanism benefiting both stability and transfer.

Third, reward function design significantly influences learned behavior. A simple steering penalty produces smoother control at modest performance cost, demonstrating a controllable speed–smoothness trade-off.

Fourth, classical baselines, while limited in absolute performance, provide safe and predictable behavior that outperforms under-trained RL agents, underscoring the importance of sufficient training budgets.

Future work could explore multi-track curriculum training, domain randomization, hierarchical RL for strategic racing decisions, and transfer from simulation to physical F1TENTH hardware.

REFERENCES

- [1] Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. arXiv:1707.06347.
- [2] Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft Actor-Critic: Off-Policy Maximum Entropy Deep RL with a Stochastic Actor. ICML 2018.
- [3] Fujimoto, S., Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods. ICML 2018.
- [4] Koirala, P. & Fleming, C. (2024). F1tenth Autonomous Racing With Offline RL Methods. arXiv:2408.04198.
- [5] F1TENTH Gym. F1TENTH Autonomous Racing Simulator. https://github.com/f1tenth/f1tenth_gym
- [6] Bosello, M. f1tenth-RL: DQN with LiDAR on F1TENTH. GitHub.
- [7] FT-Autonomous. F1Tenth Gym with PPO. GitHub.

- [8] Coulter, R.C. (1992). Implementation of the Pure Pursuit Path Tracking Algorithm. CMU-RI-TR-92-01.
- [9] Sutton, R.S. & Barto, A.G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- [10] F1TENTH Racetracks Dataset. https://github.com/f1tenth/f1tenth_racetracks